Отчет по лабораторной работе №7

Дисциплина: Архитектура ЭВМ

Тема: Команды безусловного и условного переходов в NASM. Программирование ветвлений.

Студент: А н ь с я о о н ь

Группа: НПИ-01-25

---

1. Цель работы

Изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием ветвлений. Знакомство с назначением и структурой файла листинга.

---

2. Выполнение работы

2.1. Реализация безусловных переходов (jmp)

Я создал каталог для седьмой лабораторной работы и файл `lab7-3.asm`. Я изменил логику переходов так, чтобы сообщения выводились в обратном порядке: №3, №2, №1.

Листинг 7.1 (lab7-1.asm):

Code snippet
```
%include 'in_out.asm'

SECTION .data
   msg1: DB 'Сообщение № 1',0
   msg2: DB 'Сообщение № 2',0
   msg3: DB 'Сообщение № 3',0

SECTION .text
   GLOBAL _start

_start:
   jmp _label3
```

```
_label1:
    mov eax, msg1
    call sprintLF
    jmp _end

_label2:
    mov eax, msg2
    call sprintLF
    jmp _label1

_label3:
    mov eax, msg3
    call sprintLF
    jmp _label2

_end:
    call quit
```

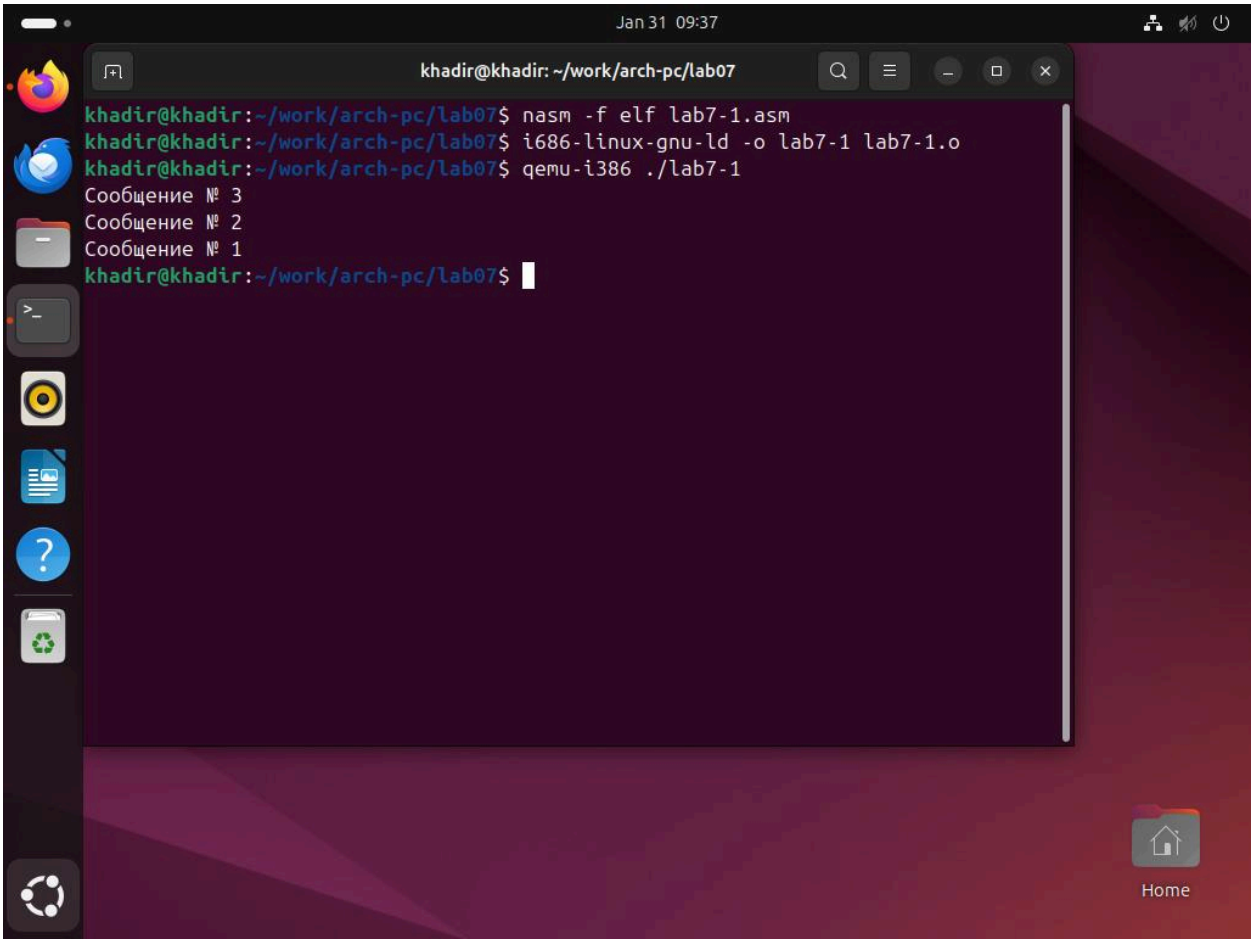

Рис. 7.1. Результат работы программы с безусловными переходами.

2.2. Определение наибольшего из трех чисел

Я создал файл `lab7-2.asm` для поиска наибольшего числа среди трех переменных (A, B, C).

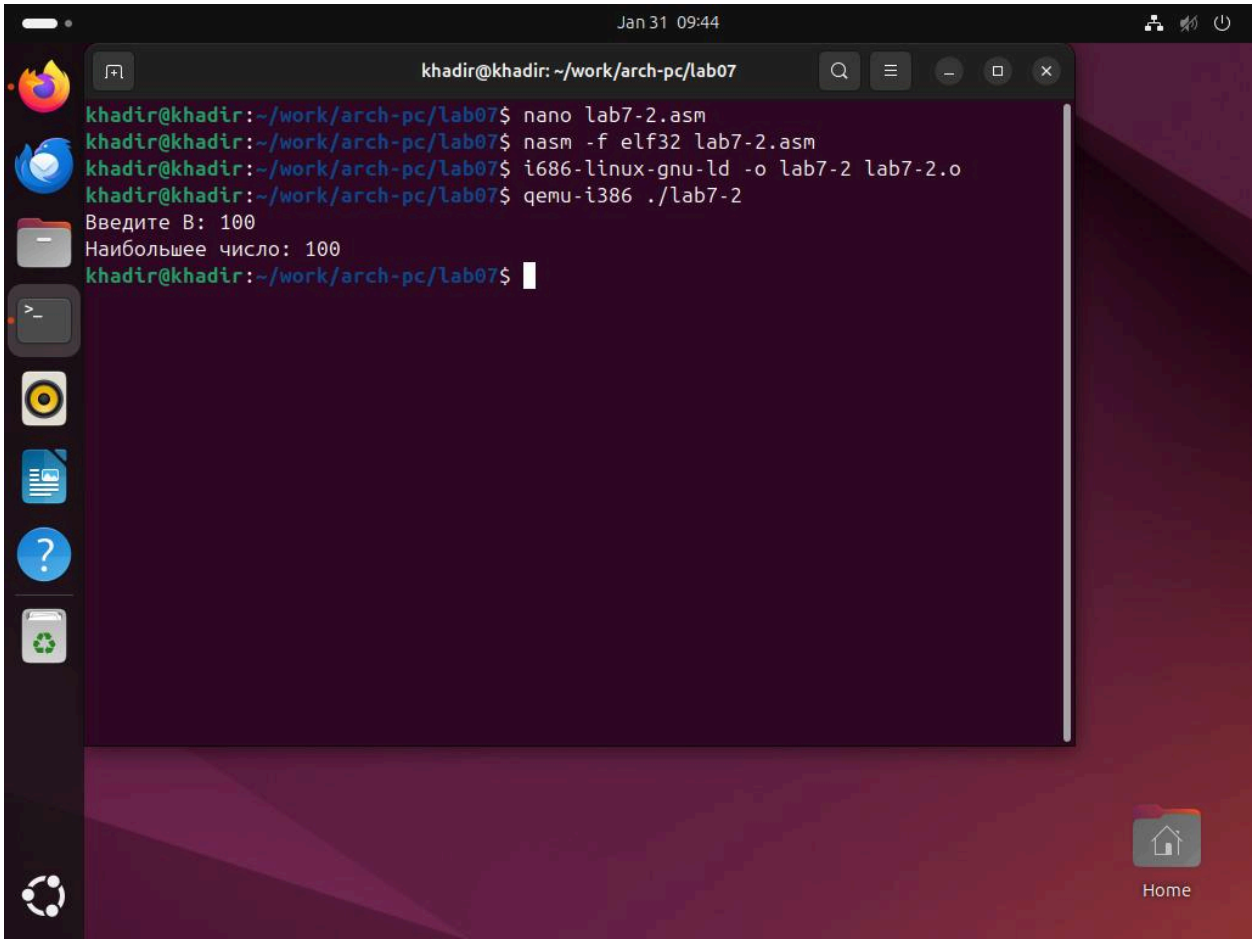

Рис. 7.2. Определение максимума из трех чисел.

2.3. Изучение файла листинга

Я создал файл листинга для программы `lab7-2.asm` с помощью ключа `-l`:

```
nasm -f elf -l lab7-2.lst lab7 -2.asm
```

```
nano lab7-2.lst
```

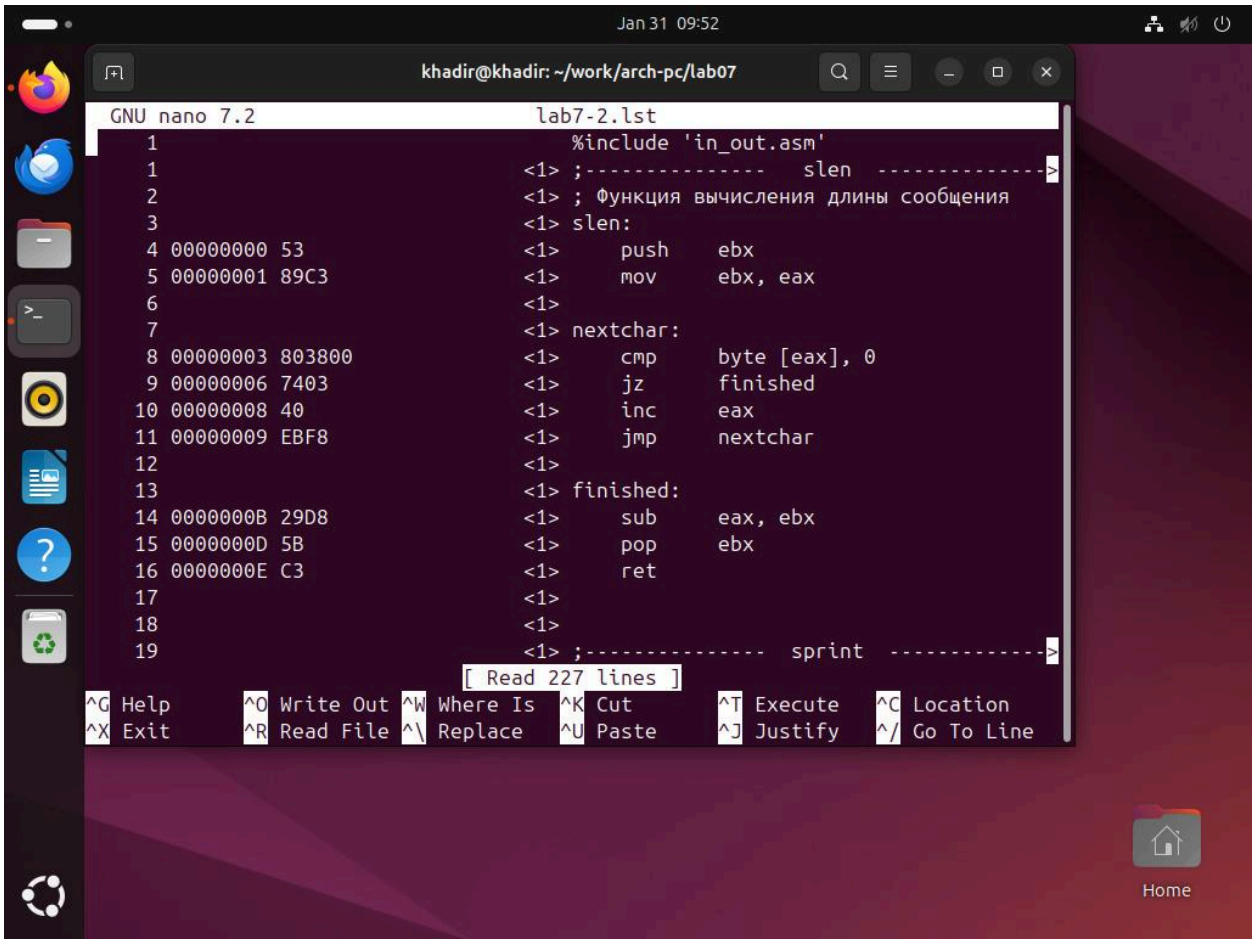

Рис. 7.3. Структура файла листинга NASM.

3. Самостоятельная работа (Вариант 12)

Задание 1: Нахождение наименьшего числа

Я написал программу для нахождения минимального из трех чисел для Варианта 12 (a=99,b=29,c=26).

Листинг 7.2 (min_find.asm):

Code snippet
```
%include 'in_out.asm'

section .data
   a dd 99
```

```
    b dd 29
    c dd 26
    msg db "Наименьшее число: ",0

section .bss
    min resb 10

section .text
    global _start

_start:
    mov ecx, [a]
    mov [min], ecx

    cmp ecx, [b]
    jl check_c
    mov ecx, [b]
    mov [min], ecx

check_c:
    mov ecx, [min]
    cmp ecx, [c]
    jl fin
    mov ecx, [c]
    mov [min], ecx

fin:
    mov eax, msg
    call sprint
    mov eax, [min]
    call iprintLF
    call quit
```

Команда: `nano lab7-2.lst`

Команда: `i686-linux-gnu-ld -o lab7-3 lab7-3.o`

Команда: `qemu-i386 ./lab7-3`

Результат: Наименьшее число: 26

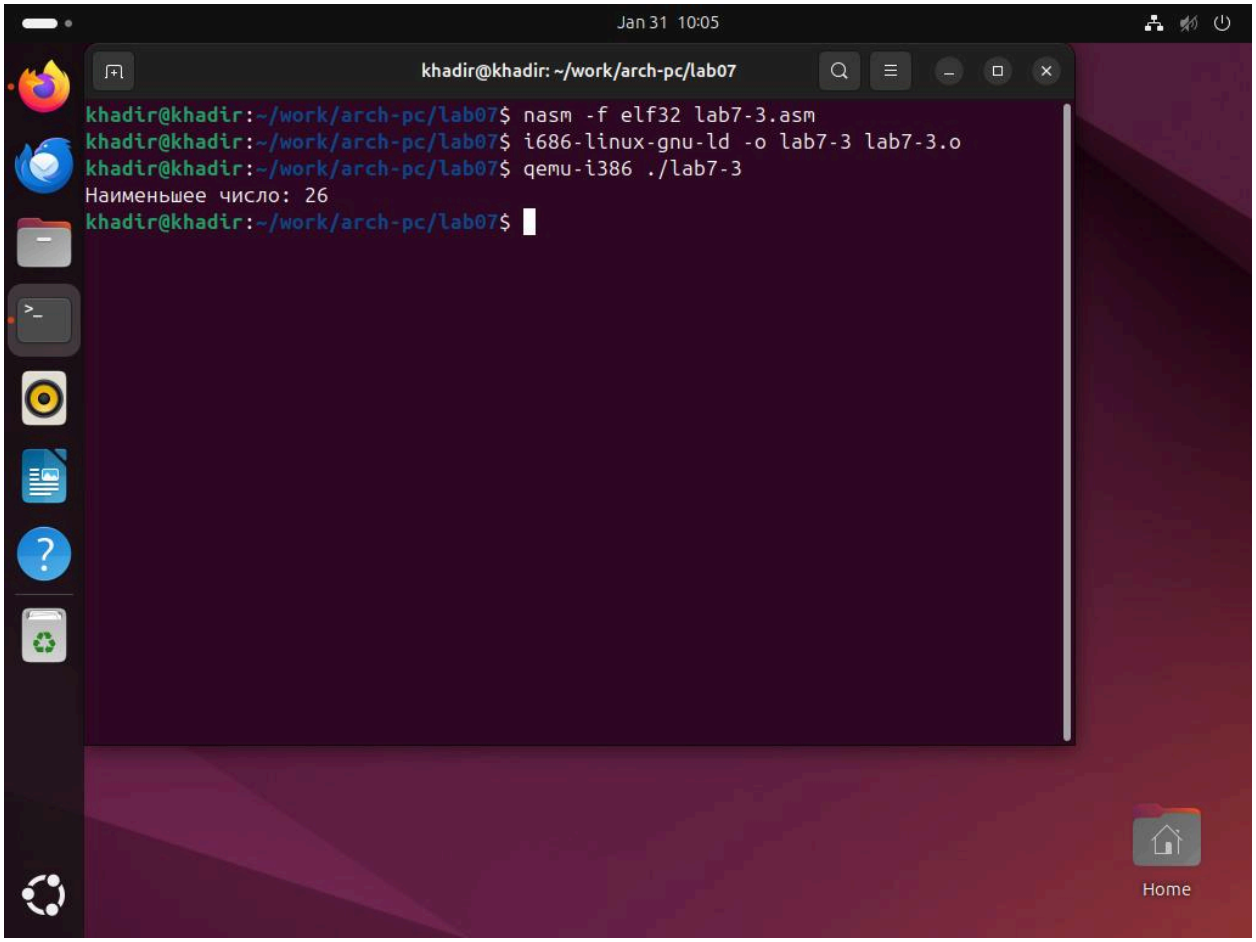

Рис. 7.4. Нахождение минимума среди чисел 99, 29 и 26.

Задание 2: Вычисление кусочной функции

Я реализовал вычисление функции для Варианта 12: f(x)=ax, если x<5 f(x)=x−5, если x≥5

Листинг 7.3 (function_calc.asm):

Code snippet

```
%include 'in_out.asm'

section .data
   msg_x db "Введите x: ",0
   msg_a db "Введите a: ",0
   msg_res db "Результат: ",0

section .bss
   x resb 10
   a resb 10
```

```
section .text
    global _start

_start:
    mov eax, msg_x
    call sprint
    mov ecx, x
    mov edx, 10
    call sread
    mov eax, x
    call atoi
    mov [x], eax

    mov eax, msg_a
    call sprint
    mov ecx, a
    mov edx, 10
    call sread
    mov eax, a
    call atoi
    mov [a], eax

    mov eax, [x]
    cmp eax, 5
    jl label_ax
    sub eax, 5
    jmp print_res

label_ax:
    mov ebx, [a]
    mul ebx

print_res:
    mov edi, eax
    mov eax, msg_res
    call sprint
    mov eax, edi
    call iprintLF
    call quit
```

Команда: `nasm -f elf32 lab7-4.asm`

Команда: `i686-linux-gnu-ld -o lab7-4 lab7-4.o`

Команда: `qemu-i386 ./lab7-4`

Результат: Для проверки корректности работы программы были проведены два теста, соответствующие обеим ветвям функции:

1. Тест №1 (x<5):
    - Ввод: x=3,a=7
    - Ожидаемый результат: f(3)=3×7=21
    - Фактический результат: 21
2. Тест №2 (x≥5):
    - Ввод: x=6,a=4
    - Ожидаемый результат: f(6)=6−5=1
    - Фактический результат: 1

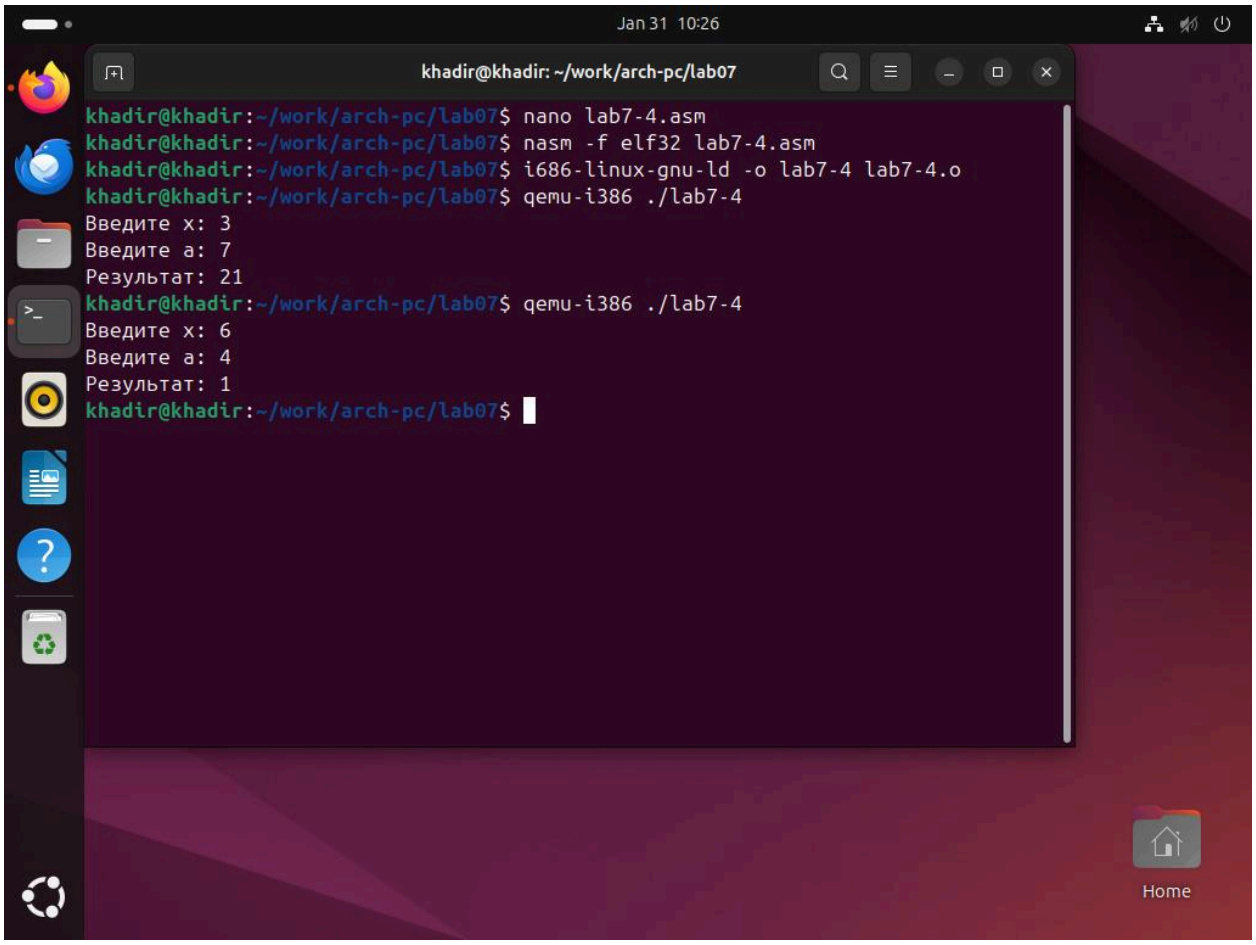

Рис. 7.5. Вычисление функции для двух наборов данных.

4. Ответы на контрольные вопросы

1. Для чего нужен файл листинга NASM? В чём его отличие от текста программы?
Файл листинга нужен для отладки. В отличие от обычной программы, он показывает не

только код, но и адреса в памяти, а также машинный код (нули и единицы в шестнадцатеричном виде).

2. Каков формат файла листинга NASM? Из каких частей он состоит? Строка листинга состоит из четырех частей: номер строки, адрес (смещение), машинный код и сам текст программы.

3. Как в программах на ассемблере можно выполнить ветвление? Ветвление делается с помощью команды сравнения `cmp` и команд перехода (например, `jne` или `jz`). Это заставляет процессор прыгать в разные части кода.

4. Какие существуют команды безусловного и условных переходов в языке ассемблера?

- **Безусловный:** `jmp` (прыгает всегда).
- **Условные:** `je` (если равно), `jne` (если не равно), `jl` (если меньше), `jg` (если больше).

5. Опишите работу команды сравнения `cmp`. Команда `cmp` вычитает второе число из первого. Она не сохраняет результат, но меняет "флаги" (индикаторы) в процессоре. На основе этих флагов работают переходы.

6. Каков синтаксис команд условного перехода? Сначала идет мнемоника (название команды), а затем метка, на которую нужно перейти. Например: `je _label1`.

7. Пример использования команды сравнения и перехода: Инструкция `cmp eax, ebx` сравнивает значения в регистрах, а следующая за ней команда `jl _label` выполняет переход на метку, если первое значение меньше второго.

8. Какие флаги анализируют команды условного перехода? Они смотрят на флаг нуля (**ZF**), флаг знака (**SF**), флаг переноса (**CF**) и флаг переполнения (**OF**).

5. Вывод

В ходе выполнения работы я изучил команды переходов в NASM. Я научился использовать логику ветвления для решения математических задач и анализа условий. Также я познакомился с файлом листинга, что помогло мне увидеть связь между кодом на ассемблере и машинными инструкциями.